

**POSTS AND TELECOMMUNICATIONS
INSTITUTE OF TECHNOLOGY**

FACULTY OF INFORMATION TECHNOLOGY



Software Architecture and Design

Tiểu luận V1

Class: E22CNPM2

Project group: 05

Project name: Tiểu luận v1

Phạm Thành Hùng – B22DCAT13

Chương 1: Sự Tiến hóa của Kiến trúc Phần mềm từ Monolithic đến Microservices và Phương pháp Domain-Driven Design

Sự phát triển vũ bão của ngành công nghiệp phần mềm và bối cảnh chuyển đổi số đã đặt ra những yêu cầu khắt khe về tính sẵn sàng, khả năng chịu lỗi và năng lực mở rộng vô hạn của các hệ thống thông tin. Các nền tảng thương mại điện tử (E-Commerce) hiện đại không chỉ phải phục vụ hàng triệu giao dịch đồng thời mà còn phải liên tục cập nhật tính năng mới mà không làm gián đoạn trải nghiệm người dùng. Trong bối cảnh này, quá trình dịch chuyển từ kiến trúc khối đơn (Monolithic Architecture) sang kiến trúc vi dịch vụ (Microservices Architecture) đã trở thành một xu hướng tất yếu và mang tính chiến lược.¹ Việc áp dụng các triết lý thiết kế hướng miền (Domain-Driven Design - DDD) cung cấp một khung lý thuyết vững chắc để phân tách các hệ thống phức tạp thành những cấu phần vi dịch vụ độc lập, gắn liền với các ranh giới nghiệp vụ thực tiễn.³

1.1 Phân tích Chuyên sâu Kiến trúc Monolithic

Kiến trúc Monolithic là mô hình truyền thống trong đó toàn bộ ứng dụng phần mềm được xây dựng, biên dịch và triển khai như một khối thống nhất duy nhất. Đặc trưng của hệ thống này là tất cả các thành phần logic như giao diện người dùng (Presentation Layer), logic nghiệp vụ (Business Logic Layer) và lớp truy cập dữ liệu (Data Access Layer) đều được gộp chung vào một cơ sở mã (codebase) và thường kết nối đến một cơ sở dữ liệu quan hệ trung tâm.¹

Ở giai đoạn đầu của vòng đời sản phẩm, khi xây dựng phiên bản Khả thi Tối thiểu (Minimum Viable Product - MVP), Monolithic mang lại nhiều lợi thế rõ rệt. Nhóm phát triển có thể dễ dàng kiểm thử toàn bộ ứng dụng (end-to-end testing), luồng thực thi hàm diễn ra trong cùng một không gian bộ nhớ (in-memory calls) giúp tối ưu hóa độ trễ cục bộ, và việc triển khai (deployment) chỉ đơn giản là sao chép một tệp thực thi duy nhất lên máy chủ.¹

Tuy nhiên, khi quy mô hệ thống phình to, kiến trúc này bộc lộ những điểm yếu chí mạng. Tính liên kết chặt chẽ (tightly coupled) khiến một thay đổi nhỏ ở mô-đun này có thể gây ra hiệu ứng domino làm hỏng chức năng của mô-đun khác.¹ Nếu mô-đun xử lý thanh toán gặp lỗi rò rỉ bộ nhớ, nó sẽ kéo theo sự sụp đổ của toàn bộ tiến trình ứng dụng, làm giảm nghiêm trọng tính sẵn sàng của hệ thống.⁷ Hơn nữa, khả năng mở rộng (scalability) của Monolithic bị giới hạn theo chiều dọc; để đáp ứng lưu lượng truy cập tăng đột biến ở một chức năng cụ thể (như tìm kiếm sản phẩm), các

kỹ sư buộc phải sao chép và nhân bản toàn bộ khối ứng dụng, gây lãng phí tài nguyên phần cứng nghiêm trọng.⁷

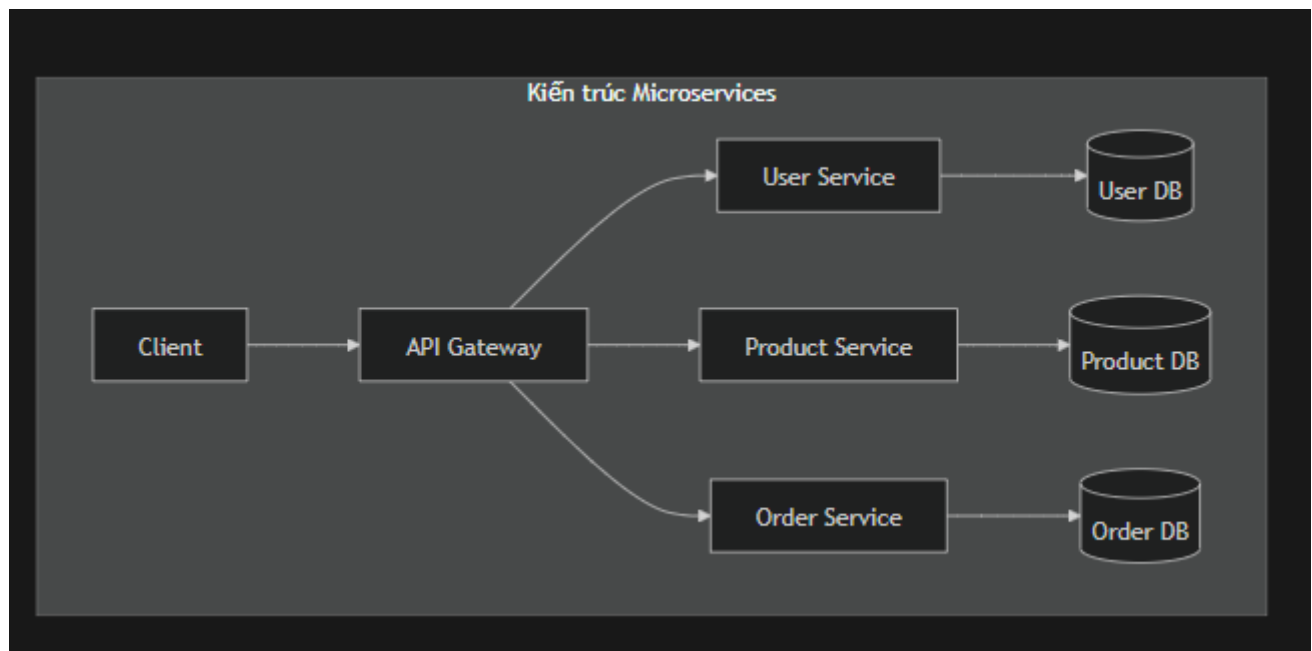
1.2 Sự Trỗi dậy của Kiến trúc Microservices

Kiến trúc Microservices giải quyết các nút thắt của Monolithic bằng cách phân rã ứng dụng thành một tập hợp các dịch vụ nhỏ, phân tán và hoạt động hoàn toàn độc lập. Mỗi vi dịch vụ chịu trách nhiệm cho một chức năng nghiệp vụ cụ thể (single responsibility), sở hữu cơ sở dữ liệu riêng biệt và giao tiếp với nhau thông qua các giao thức mạng chuẩn như HTTP/REST, gRPC hoặc thông qua các nền tảng môi giới thông điệp (Message Brokers) như Kafka, RabbitMQ.¹

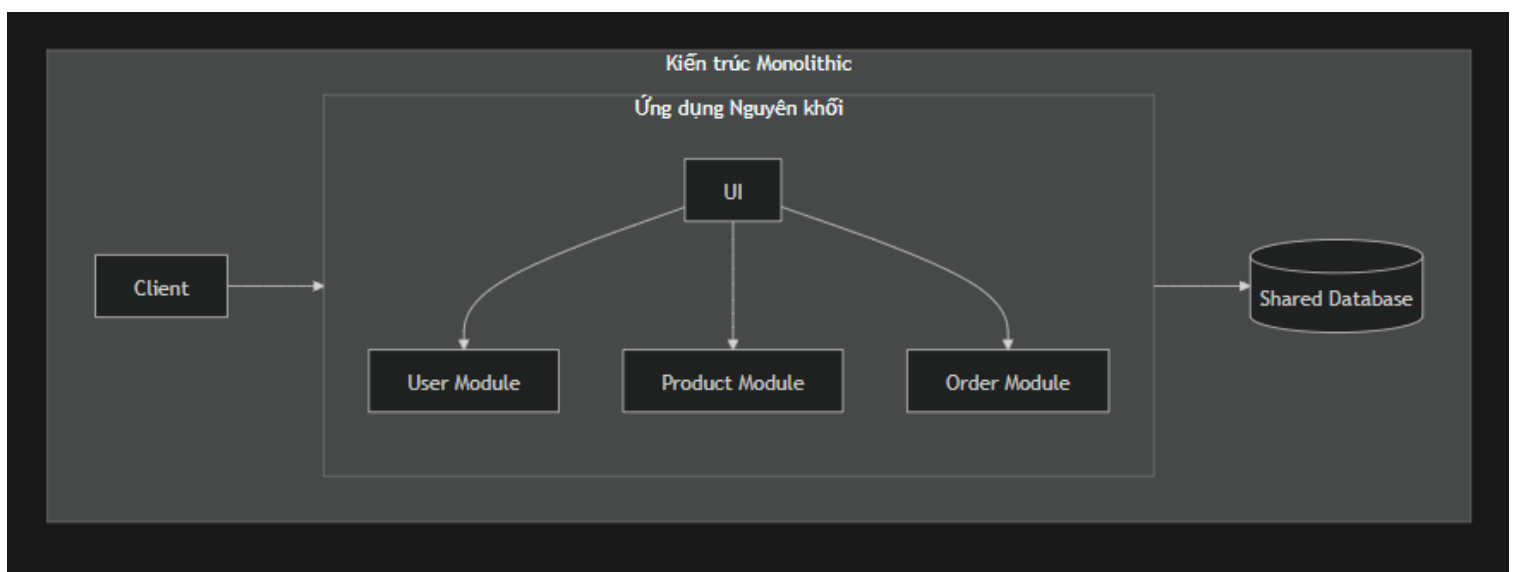
Sự phân tách này mang lại sự linh hoạt ở cấp độ kiến trúc. Các nhóm phát triển có thể làm việc song song trên các dịch vụ khác nhau bằng các ngôn ngữ lập trình đa dạng (Polyglot Programming) và công nghệ lưu trữ dữ liệu tối ưu nhất cho từng bài toán cụ thể (Polyglot Persistence).¹ Nếu dịch vụ giỏ hàng chịu tải cao trong các dịp khuyến mãi, chỉ duy nhất dịch vụ đó cần được cấp phát thêm tài nguyên (scale-out) mà không ảnh hưởng đến các thành phần khác.⁵ Khả năng cô lập lỗi (fault isolation) cũng là một thế mạnh tuyệt đối; sự cố sập cơ sở dữ liệu của dịch vụ giao hàng sẽ không làm tê liệt toàn bộ quy trình duyệt web hay thanh toán của khách hàng.⁸ Tuy vậy, Microservices cũng đưa ra những thách thức mới về tính phức tạp trong quản trị mạng, xử lý giao dịch phân tán và giám sát hệ thống.¹

1.3 Hình ảnh So sánh Hai Phương pháp Thiết kế Phần mềm

Biểu đồ kiến trúc dưới đây trực quan hóa sự khác biệt cơ bản về cấu trúc liên kết và luồng dữ liệu giữa hệ thống Monolithic và Microservices. Trong Monolithic, mọi mô-đun chia sẻ một bộ nhớ và một cơ sở dữ liệu. Ngược lại, Microservices phân phối xử lý qua cổng API Gateway, với mỗi dịch vụ được đóng gói cùng kho lưu trữ dữ liệu riêng.⁴



Sự khác biệt giữa hai kiến trúc không chỉ nằm ở khía cạnh phần mềm mà còn ở cách thức vận hành và mở rộng hạ tầng công nghệ. Bảng dưới đây cung cấp sự so sánh chi tiết giữa hai cách tiếp cận:



Tiêu chí Đánh giá	Monolithic Architecture	Microservices Architecture
Cấu trúc Tổ chức (Structure)	Khối duy nhất, chia sẻ chung cơ sở mã (codebase) ¹⁰	Tập hợp các dịch vụ phân tán, cơ sở mã độc lập ⁶
Quản trị Dữ liệu (Database)	Chia sẻ một CSDL trung tâm (Shared DB) ¹	Mỗi dịch vụ làm chủ một CSDL riêng biệt ¹
Tính Liên kết (Coupling)	Cao (Tightly Coupled), thay đổi ảnh hưởng toàn cục ¹	Thấp (Loosely Coupled), giao tiếp qua API ¹
Triển khai (Deployment)	Biên dịch và triển khai lại toàn bộ dự án ⁵	Triển khai độc lập cho từng vi dịch vụ cụ thể ¹
Khả năng Mở rộng (Scaling)	Scale toàn bộ ứng dụng (kém hiệu quả tài nguyên) ¹	Scale độc lập từng dịch vụ đang gặp nghẽn cổ chai ⁷
Cô lập Sự cố (Fault Isolation)	Kém, lỗi một mô-đun có thể sụp đổ toàn bộ ¹	Tốt, lỗi được giới hạn trong ranh giới dịch vụ ⁸

1.4 Domain-Driven Design (DDD) làm Nền tảng Phân rã

Để chuyển đổi thành công từ một yêu cầu kinh doanh phức tạp sang một kiến trúc vi dịch vụ mạch lạc, các kỹ sư cần một phương pháp luận chuẩn xác để tránh việc chia cắt hệ thống một cách tùy tiện. Domain-Driven Design (DDD) chính là chiếc chìa khóa định hình ranh giới vi dịch vụ thông qua việc đồng bộ hóa mô hình phần mềm với các miền nghiệp vụ thực tiễn (business domains).¹

Thay vì thiết kế hệ thống quanh các bảng cơ sở dữ liệu, DDD khởi đầu bằng việc xây dựng một Ngôn ngữ Chung (Ubiquitous Language) giữa các chuyên gia nghiệp vụ và kỹ sư phát triển. Từ đó, DDD thiết lập các khái niệm nòng cốt:

- **Entity (Thực thể):** Những đối tượng được định nghĩa thông qua một định danh duy nhất (ID) bất biến theo thời gian. Sự thay đổi trạng thái của Entity được theo dõi liên tục, ví dụ như hồ sơ User hay Product.¹
- **Value Object (Đối tượng Giá trị):** Những thuộc tính không yêu cầu định danh mà được nhận diện qua chính giá trị nội tại của chúng. Các đối tượng này có tính bất biến (immutable), ví dụ như Money (số tiền và loại tiền tệ) hay Address (tọa độ và chuỗi địa chỉ).¹
- **Aggregate (Tập hợp):** Nhóm các Entity và Value Object gắn kết logic với nhau thành một đơn vị duy nhất để đảm bảo tính nhất quán của dữ liệu. Mỗi Aggregate được điều khiển bởi một Aggregate Root. Bất kỳ sự thay đổi nào đối với các thành phần con đều phải được ủy quyền qua Root này (ví dụ: thực thể Order sẽ kiểm soát OrderItem).¹
- **Bounded Context (Ngữ cảnh Giới hạn):** Đây là ranh giới khái niệm xác định ngữ nghĩa rõ ràng của một mô hình nghiệp vụ. Một thuật ngữ có thể mang ý nghĩa khác nhau tùy thuộc vào Bounded Context. Nguyên lý vàng trong thiết kế vi dịch vụ là: "Mỗi Bounded Context lý tưởng nên được ánh xạ thành một Microservice độc lập".¹

1.5 Case Study Thực tiễn: Phân rã Hệ thống Y tế (Healthcare System)

Để minh chứng cho sức mạnh của DDD trong việc xử lý các bài toán nghiệp vụ phức tạp, một hệ thống thông tin bệnh viện (Healthcare Management System) được phân tích và phân rã thành các miền nghiệp vụ cụ thể.¹⁵ Trong hệ thống chăm sóc sức khỏe, dữ liệu thường gắn với tiêu chuẩn quốc tế như FHIR (Fast Healthcare Interoperability Resources). Cách tiếp cận Phân rã theo Tài nguyên (Resource-Driven Domain Decomposition) chỉ ra rằng việc gộp chung chức năng quản lý danh tính, lịch trình và tài chính là thiết kế sai lầm.¹⁶

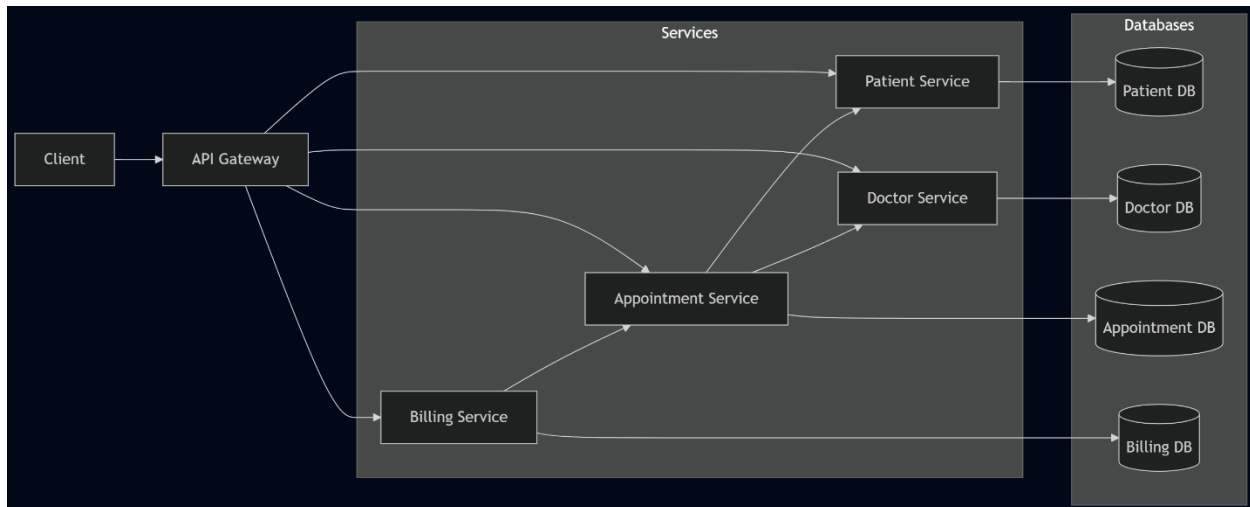
Hệ thống được bóc tách rành mạch thành 4 Bounded Contexts cốt lõi, từ đó hình thành 4 Microservices chuyên biệt¹:

1. **Patient Context (Patient Service):** Quản lý toàn bộ thông tin định danh, lịch sử y tế và hồ sơ bệnh nhân. Dịch vụ này thao tác trực tiếp trên các tài nguyên FHIR tương ứng như Patient, Person.¹⁶
2. **Doctor/Practitioner Context (Doctor Service):** Lưu trữ thông tin định danh, ca trực, đánh giá hiệu suất, và chuyên môn của đội ngũ y bác sĩ.¹
3. **Scheduling Context (Appointment Service):** Trọng tâm của việc điều phối thời gian. Dịch vụ này xử lý thuật toán tìm kiếm khung giờ rảnh, điều phối ca khám giữa bác sĩ và bệnh nhân, sử dụng tài nguyên Appointment và Slot.¹⁶

4. **Financial/Billing Context (Billing Service):** Chịu trách nhiệm hoàn toàn về dòng tiền, quản lý chi phí dịch vụ, hóa đơn điện tử và tương tác với các hệ thống bảo hiểm.¹⁷

Hình: Sơ đồ Phân rã Ngữ cảnh (Context Map / Decomposition Diagram) của Hệ thống Healthcare

Sơ đồ dưới đây minh họa việc chia rẽ chức năng (Decomposition) và cách các vi dịch vụ tương tác (Inter-service Communication). Thay vì gọi trực tiếp cơ sở dữ liệu của nhau, Appointment Service sẽ giao tiếp với Doctor Service qua REST API để lấy ca trực, trong khi Billing Service sẽ nhận tín hiệu hoàn thành phiên khám qua kiến trúc hướng sự kiện (Event-driven Architecture).¹⁶



Sự phân rã này đảm bảo rằng mỗi dịch vụ có thể được bảo trì, cập nhật và mở rộng độc lập. Dịch vụ Patient Service có thể cần lưu trữ lượng lớn hồ sơ phi cấu trúc dưới dạng cơ sở dữ liệu tài liệu (Document DB), trong khi Billing Service bắt buộc phải triển khai trên cơ sở dữ liệu quan hệ (Relational DB) để đảm bảo tính tuân tự (ACID) cho giao dịch tài chính.¹ Phương pháp luận này sẽ tiếp tục được triển khai triệt để trong chương tiếp theo dành riêng cho hệ thống E-Commerce.

Chương 2: Phân tích, Thiết kế Kiến trúc và Mô hình Hóa Dữ liệu cho Hệ thống E-Commerce

Xây dựng nền tảng E-Commerce vi dịch vụ đòi hỏi việc phân tích tỉ mỉ các yêu cầu chức năng (Functional Requirements) như quản lý danh mục sản phẩm, theo dõi giỏ hàng, đặt hàng, xử lý thanh toán, và các yêu cầu phi chức năng (Non-functional Requirements) như khả năng chịu tải cao, tính bảo mật và tính sẵn sàng liên tục.¹ Áp

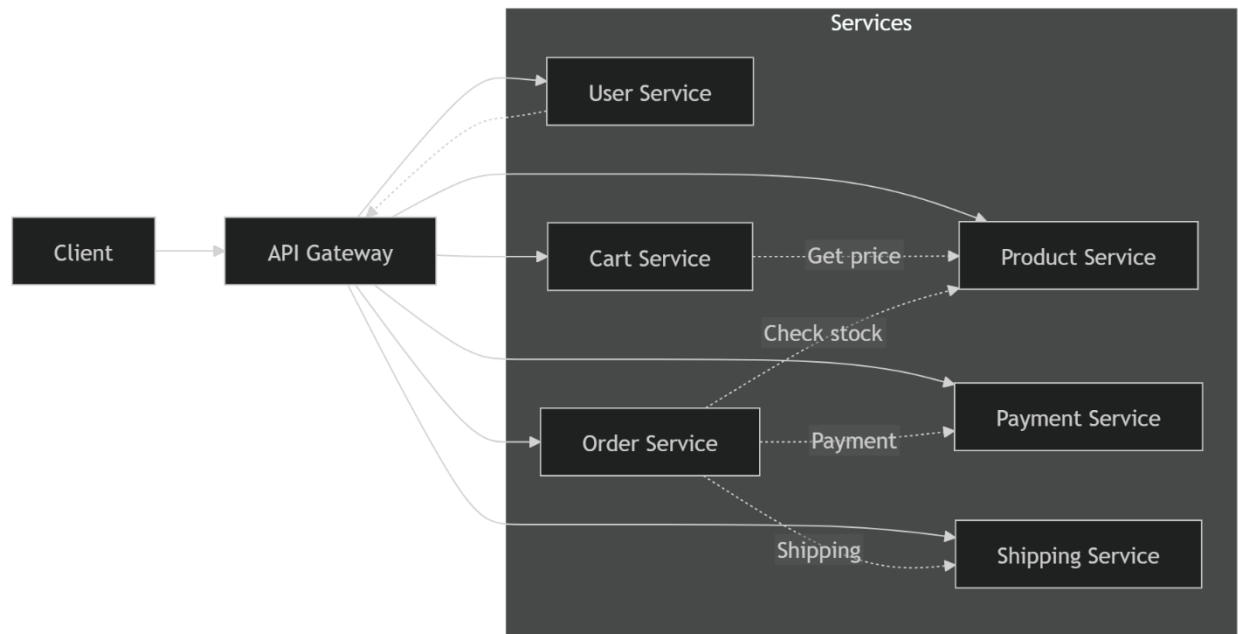
dụng nguyên lý DDD, hệ thống E-Commerce được phân rã thành các dịch vụ độc lập nhằm đảm bảo khả năng phối hợp nhịp nhàng trong điều kiện lưu lượng truy cập lớn.³

2.1 Hình Phân rã Hệ thống E-Commerce thành các Microservice

Toàn bộ nền tảng thương mại điện tử được bóc tách thành sáu Bounded Contexts (ngữ cảnh giới hạn), tương đương với sáu khối vi dịch vụ cốt lõi.¹

1. **User Service:** Chịu trách nhiệm quản lý định danh người dùng (Khách hàng, Nhân viên, Quản trị viên), thực thi các cơ chế xác thực và phân quyền (Role-Based Access Control - RBAC).¹
2. **Product Service (Catalog & Inventory):** Quản lý hệ thống phân loại hàng hóa đa miền (Sách, Điện tử, Thời trang). Dịch vụ này cũng theo dõi lượng hàng tồn kho theo thời gian thực.¹
3. **Cart Service:** Duy trì trạng thái giỏ hàng tạm thời của khách hàng. Đặc điểm của dịch vụ này là cần thao tác ghi/xóa liên tục với tốc độ cao.¹
4. **Order Service:** Trái tim của hệ thống giao dịch, quản lý toàn bộ vòng đời của một đơn hàng kể từ khi khởi tạo, chờ thanh toán, đến khi đóng gói.¹
5. **Payment Service:** Tích hợp với các cổng thanh toán bên thứ ba, quản lý trạng thái luân chuyển tiền tệ (Pending, Success, Failed) và hoàn tiền.¹
6. **Shipping Service:** Theo dõi luồng logistics vận chuyển vật lý, cập nhật trạng thái giao nhận và quản lý địa chỉ.¹

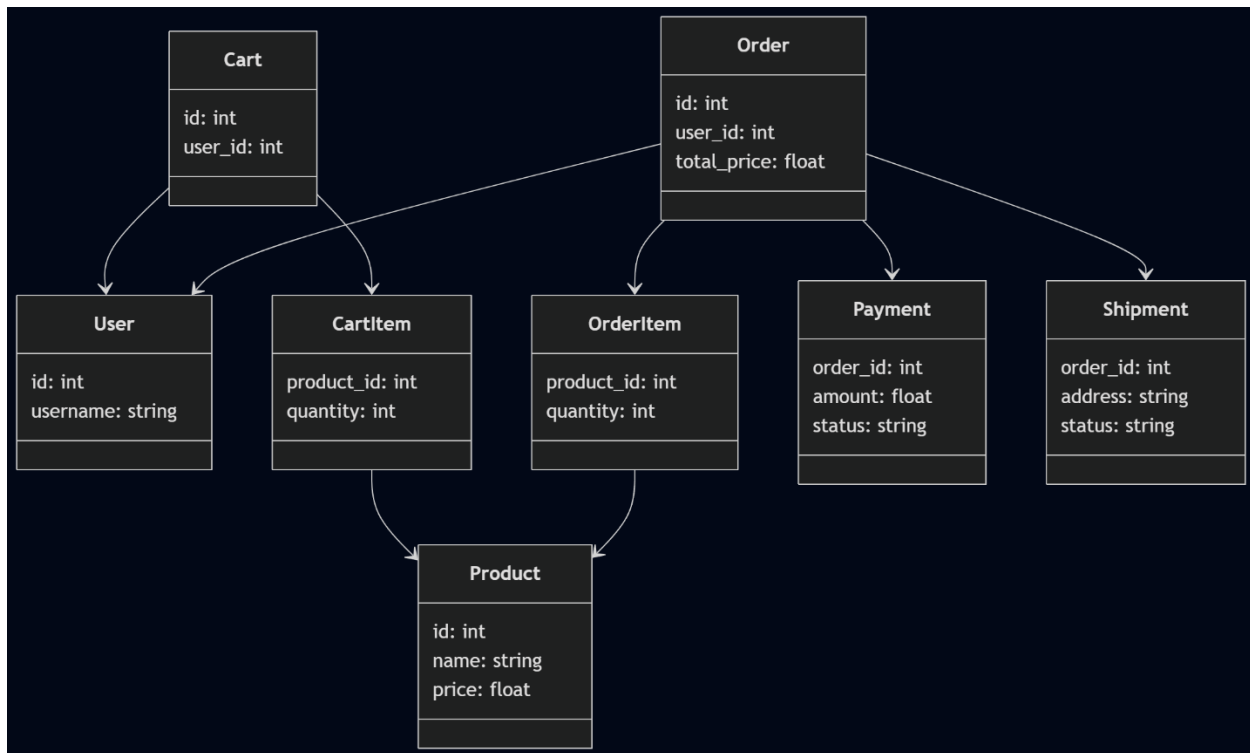
Hình: Sơ đồ Kiến trúc Phân rã Dịch vụ (Service Decomposition Diagram)



2.2 Biểu đồ Lớp Thiết kế (Class Diagram) Toàn diện cho Từng Dịch vụ

Quá trình mô hình hóa hệ thống bắt đầu bằng việc xác định cấu trúc Lớp (Class), thuộc tính (Attributes) và phương thức (Operations) bên trong từng dịch vụ.²⁴ Việc thiết kế phải tuân thủ nghiêm ngặt nguyên tắc cô lập: một Aggregate của dịch vụ này không được chứa cấu trúc trực tiếp trở vào Aggregate của dịch vụ khác. Sự liên kết giữa chúng chỉ được biểu diễn qua Định danh (ví dụ: `user_id` thay vì đối tượng `User` trực tiếp).¹

Dưới đây là Biểu đồ Lớp Tổng thể (Unified UML Class Diagram) minh họa chi tiết cấu trúc hướng đối tượng của sáu dịch vụ riêng biệt. Đặc biệt, trong **Product Service**, tính đa hình (Polymorphism) và kế thừa (Inheritance) được áp dụng để phân tách các loại mặt hàng mang đặc tính dữ liệu khác nhau.¹



Phân tích thiết kế Lớp:

- **Kế thừa trong Product:** Việc phân cấp đối tượng Book, Electronics, Fashion kế thừa từ lớp Product gốc cho phép hệ thống dễ dàng mở rộng khi xuất hiện thêm các ngành hàng mới mà không làm phá vỡ logic tính giá hay trừ kho ở lớp cha.¹
- **Tham chiếu định danh (ID Reference):** Trong lớp Order, thay vì khởi tạo một đối tượng tham chiếu User user, lớp này chỉ lưu trữ int user_id. Khớp nối lỏng lẻo này bảo vệ dịch vụ Đặt hàng khỏi những thay đổi cấu trúc bên trong dịch vụ Người dùng.²⁴

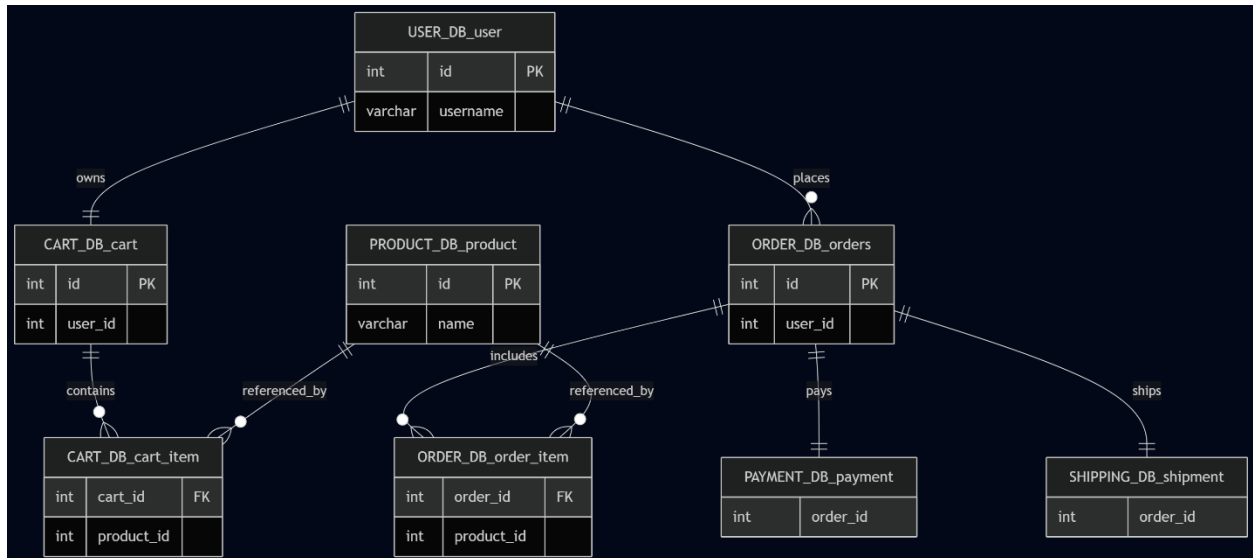
2.3 Mô hình Dữ liệu (Data Model) Chuyển đổi từ Biểu đồ Lớp

Mỗi Microservice sở hữu một kiến trúc lưu trữ riêng (Database-per-Service). Dữ liệu cấu trúc từ biểu đồ lớp được dịch chuyển thành Mô hình Thực thể - Mối quan hệ (Entity-Relationship Diagram - ERD).¹

- **MySQL cho User, Cart, Order, Payment, Shipping:** Hệ quản trị cơ sở dữ liệu quan hệ MySQL phù hợp tuyệt đối cho các dịch vụ yêu cầu tính bảo mật xác thực và nhất quán ACID (Atomicity, Consistency, Isolation, Durability) đối với các giao dịch tài chính.¹
- **PostgreSQL cho Product Service:** Mặt khác, dịch vụ Sản phẩm lại chứa thông tin rất đa dạng (ví dụ kích cỡ, màu sắc của thời trang khác với số ISBN

của sách). PostgreSQL, với khả năng truy vấn kiểu dữ liệu JSONB cực kỳ mạnh mẽ, được lựa chọn để lưu trữ các trường dữ liệu tùy biến mà không cần phải thay đổi cấu trúc Schema (cột) liên tục.¹

Hình: Biểu đồ Mô hình Dữ liệu (ERD) Vật lý của Toàn hệ thống



Ghi chú: Các liên kết khóa ngoại (Foreign Key - FK) chỉ được thiết lập mức vật lý bên trong cùng một cơ sở dữ liệu. Giữa các dịch vụ, khóa ngoại mang tính chất logic (Logical FK).

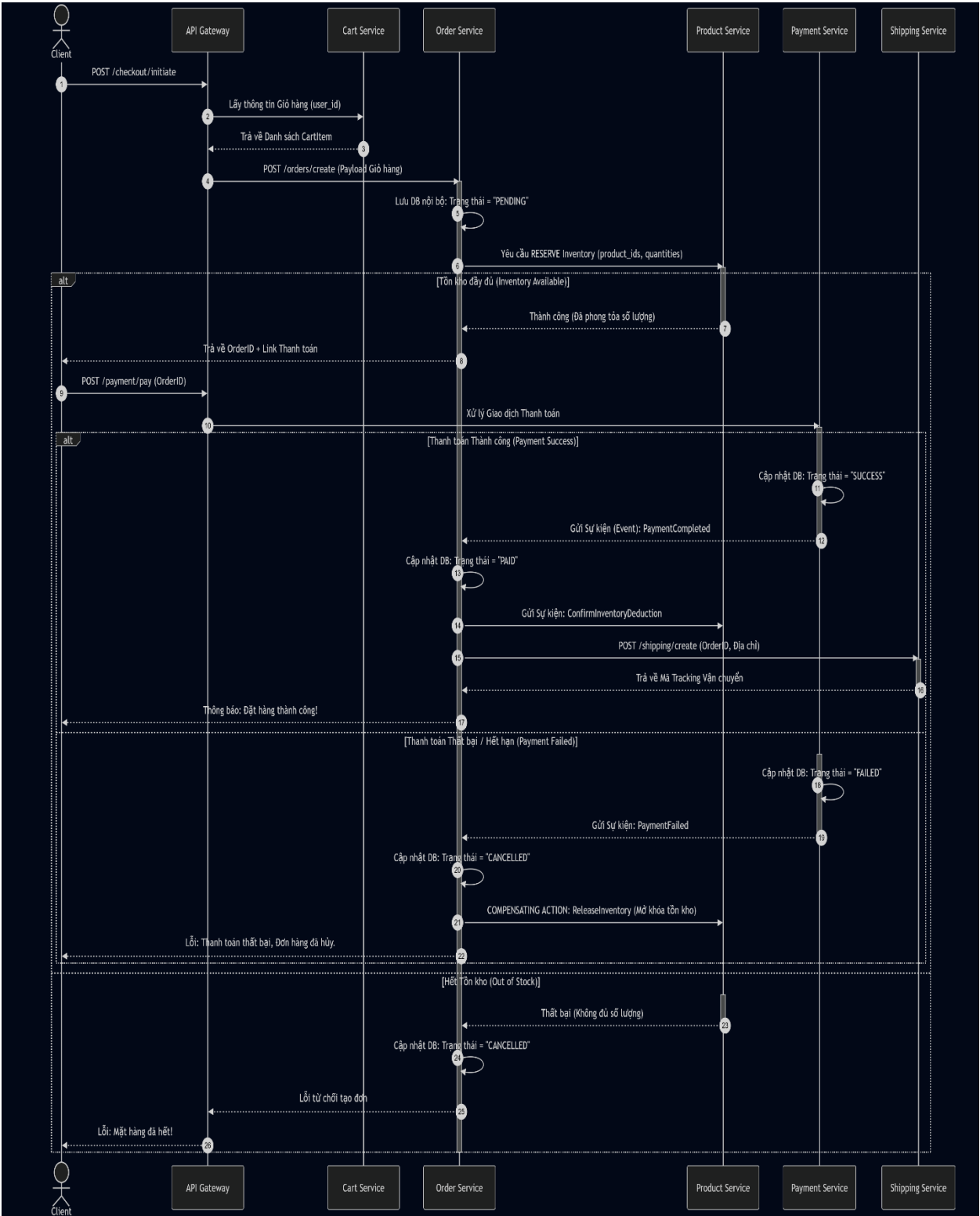
2.4 Biểu đồ Luồng Xử lý Quan trọng: Quy trình Mua hàng Phân tán (Saga Pattern)

Một trong những thách thức kỹ thuật lớn nhất của kiến trúc Microservices là đảm bảo tính toàn vẹn giao dịch (Transaction Integrity) xuyên suốt nhiều dịch vụ.²⁸ Đối với quy trình Mua hàng (Checkout), khi khách hàng bấm nút đặt hàng, thao tác này liên quan đến 4 dịch vụ: Order (Tạo đơn), Product (Trừ tồn kho), Payment (Trừ tiền) và Shipping (Vận chuyển).¹

Thay vì sử dụng giao dịch khóa hai pha (2-Phase Commit) vốn gây nghẽn cổ chai, hệ thống triển khai mô hình **Saga Pattern**. Saga quản lý giao dịch thông qua một chuỗi các sự kiện cục bộ liên tiếp. Nếu một bước trong chuỗi thất bại (ví dụ tài khoản khách hàng không đủ tiền), hệ thống sẽ kích hoạt một chuỗi các **Giao dịch Bù trừ (Compensating Transactions)** để hoàn tác các bước trước đó (trả lại tồn kho).²⁹

Luồng tương tác được phối hợp theo mô hình Biên đạo múa (Choreography) kết hợp Nhạc trưởng (Orchestration), thể hiện qua Biểu đồ Tuần tự (Sequence Diagram) chi tiết dưới đây.³

Hình: Biểu đồ Tuần tự Luồng Xử lý Đặt hàng và Thanh toán



Phân tích Luồng Xử lý:

- **Bước 5-6 (Giữ chỗ tài nguyên):** Thay vì trừ tồn kho vĩnh viễn ngay lập tức, Product Service sử dụng cơ chế "Reserve" (Giữ chỗ/Phong tỏa) để tránh hiện tượng đặt hàng trùng lặp.
- **Bước 18 (Giao dịch Bù trừ):** Đây là tinh hoa của mô hình Saga.³⁰ Khi Payment Service báo cáo giao dịch không thể thực hiện, Order Service chịu trách nhiệm gọi API hoàn tác (Rollback) giải phóng số hàng đã bị giữ chỗ tại Product Service, đưa hệ thống về lại trạng thái nhất quán cuối cùng.²⁹

Chương 3: AI Service – Tích Hợp Đồ thị Tri thức, Học Sâu và RAG Cho Hệ Sinh Thái Tư Vấn

Kiến trúc phần mềm tối ưu chỉ là nền tảng; việc cung cấp trải nghiệm mua sắm cá nhân hóa đột phá mới là đích đến của thương mại điện tử hiện đại. Để hiện thực hóa mục tiêu này, một vi dịch vụ độc lập mang tên **AI Service** được thiết kế dựa trên cấu trúc lai (Hybrid Architecture), kết hợp ba công nghệ mũi nhọn: Khai phá Dữ liệu Chuỗi bằng Mạng Nơ-ron (Deep Learning LSTM), Đồ thị Tri thức (Knowledge Graph - Neo4j) và Hệ thống Thể hệ Tăng cường Truy xuất (Retrieval-Augmented Generation - RAG).¹

Sự kết hợp này vượt qua giới hạn của hệ thống đề xuất truyền thống (chỉ dựa vào ma trận Collaborative Filtering), cung cấp khả năng hiểu sâu sắc cả về thói quen theo thời gian của người dùng lẫn mối quan hệ ngữ nghĩa của sản phẩm.³³

3.1 Khai phá Hành vi với Deep Learning: Mô hình LSTM

Khách hàng tương tác với hệ thống không phải ngẫu nhiên mà tuân theo một dòng chảy suy nghĩ nhất định (ví dụ: Xem điện thoại → Xem ốp lưng → Thêm vào giỏ hàng cáp sạc). Hành vi này là một chuỗi tuần tự thời gian. Mô hình **Long Short-Term Memory (LSTM)**, một biến thể mạnh mẽ của Mạng Nơ-ron Hồi quy (RNN), được sử dụng để lập mô hình chuỗi (Sequence Modeling).¹

LSTM giải quyết hiệu quả vấn đề "biến mất đạo hàm" (vanishing gradient) qua cấu trúc bộ nhớ tế bào (cell state) và các cổng thông tin.³⁷

- **Cơ chế hoạt động:** Ở mỗi bước thời gian t , người dùng có một hành động x_t (được chuyển đổi thành vector embedding). Mạng LSTM sử dụng **Cổng Quên (Forget Gate)** để quyết định lượng thông tin dư thừa cần loại bỏ khỏi ký ức 37.

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

- **Cổng Đầu vào (Input Gate)** và **Cổng Đầu ra (Output Gate)** sẽ cập nhật và đẩy dự đoán ra ngoài. Tại bước cuối cùng của phiên duyệt web, lớp mạng Dense (Fully Connected) sẽ xuất ra phân phối xác suất (Softmax) dự đoán mặt hàng tiếp theo khách hàng sẽ mua.¹

3.2 Lập bản đồ Mạng lưới Sản phẩm với Knowledge Graph (Neo4j)

Trong khi LSTM xử lý tính tuần tự, thì cơ sở dữ liệu đồ thị Neo4j được triển khai để giải quyết khía cạnh "không gian" của dữ liệu. Toàn bộ nền tảng thương mại điện tử được ánh xạ thành một **Đồ thị Tri thức (Knowledge Graph)** gồm hàng triệu Nút (Node - ví dụ: Khách hàng, Sản phẩm, Thương hiệu) và Cạnh (Edge/Relationship - ví dụ: BUY, VIEW, BELONGS_TO, SIMILAR_TO).¹

Hệ thống RDBMS truyền thống tỏ ra chậm chạp và tiêu tốn cực kỳ nhiều tài nguyên khi phải thực hiện các phép JOIN để quy qua nhiều bảng để tìm ra sự tương đồng. Ngược lại, Neo4j xử lý các phép duyệt đồ thị (Graph Traversal) ở tốc độ mili-giây.³⁹

Ví dụ Truy vấn Cypher Khuyến nghị dựa trên Cộng đồng: Đề gợi ý sản phẩm thay thế cho một mặt hàng đang hết kho, hệ thống áp dụng kỹ thuật lý luận đa bước (multi-hop reasoning) thông qua Cypher¹:

```
MATCH (u:User {id: 42})-->(p:Product)-->(c:Category)
```

```
MATCH (rec:Product)-->(c)
```

```
WHERE rec.stock > 0 AND NOT (u)-->(rec)
```

```
WITH rec, rand() AS random_score
```

```
ORDER BY random_score LIMIT 5
```

```
RETURN rec.id, rec.name, rec.price
```

3.3 GraphRAG: Phối hợp Vector Database và Knowledge Graph

Tính năng Chatbot Tư vấn AI yêu cầu khả năng hiểu ngôn ngữ tự nhiên. Việc sử dụng trực tiếp các Mô hình Ngôn ngữ Lớn (LLM) như GPT hay Llama tiềm ẩn rủi ro sinh hoang tưởng (hallucinations - đưa ra thông tin sản phẩm không có thực).³³ Khung **RAG (Retrieval-Augmented Generation)** giải quyết điều này bằng cách truy xuất thông tin từ cơ sở dữ liệu làm bối cảnh (context) ép buộc LLM trả lời.¹

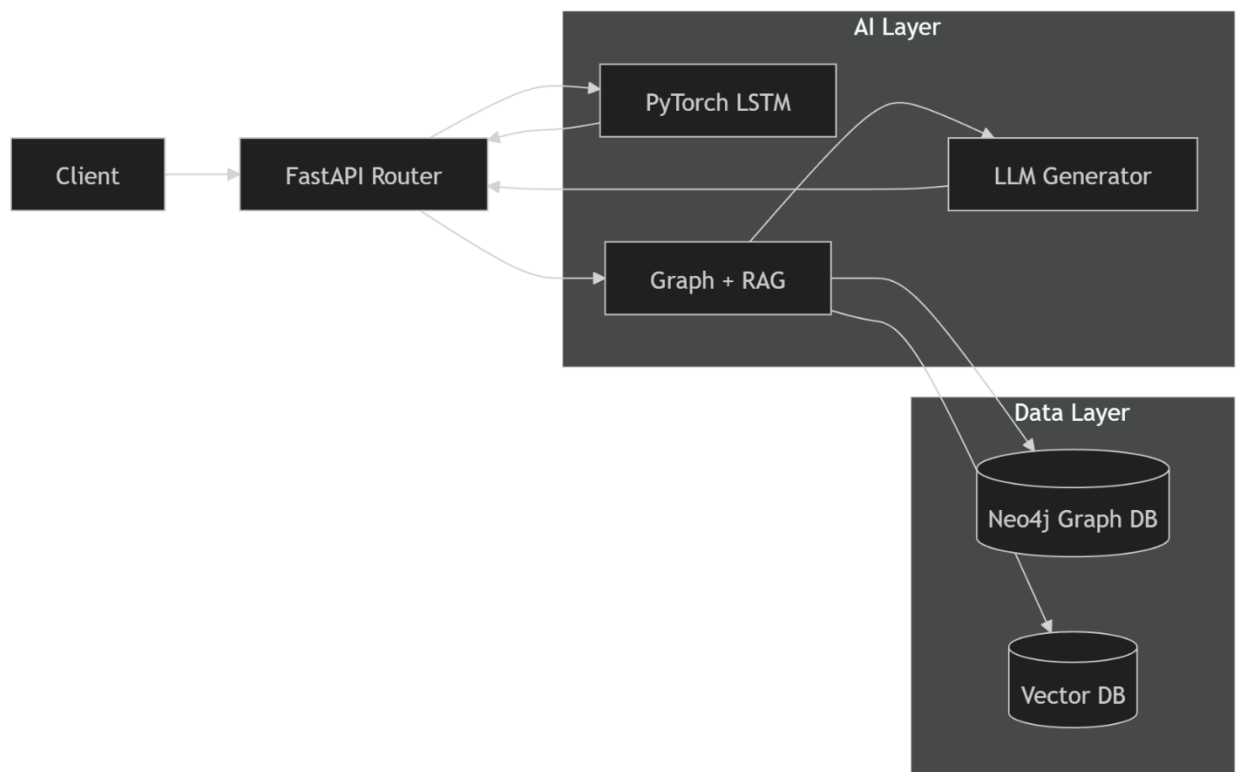
Tuy nhiên, RAG truyền thống dựa trên cơ sở dữ liệu Vector (như FAISS hay ChromaDB) chỉ đối chiếu sự tương đồng ngữ nghĩa từng mảnh văn bản cô lập, dẫn đến mất bối cảnh toàn cục (contextual continuity).³⁹ **GraphRAG** là bước tiên hóa triệt để khi kết hợp cả hai³⁴:

1. **Pipeline Khởi tạo (Ingestion):** Dữ liệu mô tả hàng hóa được đưa qua LLM để trích xuất thành các Bộ ba (Triplets) thực thể và liên kết vào Neo4j, đồng thời được nhúng vector (embedding) lưu vào ChromaDB.³⁴
2. **Pipeline Truy xuất (Retrieval):** Khi người mua hỏi "Tìm laptop gaming hãng Asus giá dưới 20 triệu", hệ thống tiến hành tìm kiếm Vector để hiểu ngữ nghĩa "gaming" và song song dùng Neo4j lọc theo quan hệ brand="Asus" và cấu trúc giá trị. Sự dung hợp (Reciprocal Rank Fusion - RRF) sinh ra nguồn tri thức chính xác nhất.³⁴
3. **Pipeline Sinh văn bản (Generation):** Bối cảnh cấu trúc này được nhồi vào Prompt cho LLM. Kết quả đầu ra là câu trả lời mượt mà, định hướng mua hàng chính xác và có thể giải thích được nguồn gốc (explainability).³³

3.4 Kiến trúc Triển khai (Deploy) AI Service

AI Service được đóng gói riêng biệt dưới dạng ứng dụng **FastAPI**, tận dụng năng lực lập trình bất đồng bộ (Async) lý tưởng cho các tính toán Machine Learning nặng và tương tác LLM.¹

Hình: Biểu đồ Kiến trúc Lai của AI Service



Mô hình toán học lai cho hệ thống Recommendation kết hợp các chiến lược để tối đa độ chính xác:

$$Final_Score = w_1 \cdot Score_{LSTM} + w_2 \cdot Score_{Graph} + w_3 \cdot Score_{RAG} .^1$$

Qua đó, AI Service cung cấp được 2 endpoint chính: Danh sách Gợi ý tức thời (/recommend) và Chatbot Tư vấn Đa lượt (/chatbot).¹

Chương 4: Hiện Thực Hóa - Demo Code, Bảo Mật và Triển Khai Hệ Thống Toàn Diện

Một thiết kế kiến trúc phân tán dù có nền tảng lý thuyết hoàn hảo cũng không thể hoạt động ổn định nếu thiếu đi quy trình triển khai cơ sở hạ tầng chuyên nghiệp. Chương này đi sâu vào việc viết cấu hình mã nguồn (Infrastructure as Code) cho API Gateway, cơ chế bảo mật Token, đóng gói tự động hóa với Docker và chuẩn hóa mạng lưới giao tiếp liên dịch vụ.¹

4.1 Cấu Hình API Gateway với Nginx

Là điểm chạm duy nhất của người dùng tới hệ thống backend, **API Gateway** che giấu toàn bộ sự phức tạp của kiến trúc lưới (mesh) bên trong. Nginx được thiết lập làm Reverse Proxy đóng vai trò định tuyến (routing) và xử lý xác thực sớm (authentication offloading).¹

Mã Demo cấu hình nginx.conf thực tế: Đoạn mã cấu hình dưới đây phân nhánh các luồng dữ liệu REST API từ Frontend trở thẳng tới vùng chứa Docker của từng dịch vụ tương ứng, đồng thời truyền các siêu dữ liệu (metadata) thiết yếu qua HTTP Header.

```

worker_processes auto;
events {
    worker_connections 2048;
}

http {
    # Khai báo các upstream trỏ đến các microservice cục bộ
    upstream user_service_pool { server user-service:8000; }
    upstream product_service_pool { server product-service:8001; }
    upstream ai_service_pool { server ai-service:8080; }

    server {
        listen 80;
        server_name api.ecommerce-demo.vn;

        # Cấu hình CORS và Header bảo mật cơ bản
        add_header X-Frame-Options "SAMEORIGIN";
        add_header X-XSS-Protection "1; mode=block";

        Ctrl+L for Agent

        # Route tối User Service (Auth/Login/Profile)
        location /api/users/ {
            proxy_pass http://user_service_pool;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        }

        # Route tối Product Service
        location /api/products/ {
            proxy_pass http://product_service_pool;
            proxy_set_header Host $host;
        }

        # Route tối Hệ sinh thái Tư vấn AI
        location /api/ai/ {
            proxy_pass http://ai_service_pool;
            proxy_read_timeout 120s; # LLM cần thời gian xử lý dài hơn
        }
    }
}

```

4.2 Thiết lập Xác thực Phi Trạng thái với JWT (JSON Web Token)

Bởi các Microservices hoạt động hoàn toàn độc lập, việc duy trì một Session/Cookie dùng chung là bất khả thi. Kiến trúc sử dụng **JWT (JSON Web Token)** để cấp phát quyền định danh phi trạng thái (Stateless Authentication).¹ Token này chứa một khối lượng payload gọn nhẹ mã hóa Base64 (bao gồm user_id, role, và exp hạn sử dụng) và được ký bằng thuật toán bí mật (HMAC SHA-256).

Demo Code - Tích hợp JWT bằng Django REST Framework (Tại User Service):

```
# settings.py cấu hình sử dụng simplejwt
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ),
    'DEFAULT_PERMISSION_CLASSES': (
        'rest_framework.permissions.IsAuthenticated',
    )
}

# urls.py cấu hình endpoint sinh Token
from django.urls import path
from rest_framework_simplejwt.views import TokenObtainPairView, TokenRefreshView
```

Mỗi khi Frontend truy vấn các dịch vụ khác (như giỏ hàng hay đơn hàng), JWT sẽ được đính kèm vào Header Authorization: Bearer <token>. Dịch vụ nhận request sẽ giải mã chữ ký cục bộ mà không cần phải gọi ngược lại DB của User Service, tối ưu hóa độ trễ cục bộ.

4.3 Giao Tiếp Liên Dịch Vụ và Kỹ thuật Bắt Lỗi (Service Communication)

Các vi dịch vụ giao tiếp với nhau chủ yếu qua môi trường mạng nội bộ, điều này đồng nghĩa với việc đối mặt với độ trễ và khả năng mất kết nối (Network Fallacy).²¹ Việc gọi API trực tiếp (Synchronous REST API) phải tuân thủ nghiêm ngặt các thực hành tốt (Best Practices) như thiết lập thời gian chờ (Timeout) và cơ chế thử lại (Retry / Circuit Breaker).

Demo Code - Giao tiếp Python Requests (Tại Order Service gọi Payment Service):

```

import requests
import logging
from requests.exceptions import Timeout, RequestException

def trigger_payment(order_id: int, amount: float, user_jwt: str) -> dict:
    url = "http://payment-service:8004/api/payment/pay/"
    payload = {"order_id": order_id, "amount": amount}
    headers = {"Authorization": f"Bearer {user_jwt}"}

    try:
        # Thiết lập timeout chặt chẽ để không treo luồng của Order Service
        response = requests.post(url, json=payload, headers=headers, timeout=5.0)
        response.raise_for_status()
        return response.json()
    except Timeout:
        logging.error(f"Timeout khi gọi Payment Service cho Order {order_id}")
        return {"status": "FAILED", "reason": "TIMEOUT"}
    except RequestException as e:
        logging.error(f"Lỗi mạng: {e}")
        return {"status": "FAILED", "reason": "NETWORK_ERROR"}

```

4.4 Docker Hóa Toàn Hệ Thống (Containerization)

Để chuẩn hóa môi trường từ khâu phát triển đến sản xuất thực tế, toàn bộ kiến trúc được bọc trong các vùng chứa (Containers) bằng công nghệ **Docker**. Mọi thư viện phần mềm, mã nguồn và tệp cấu hình được khóa cứng bên trong một Image, loại trừ triệt để tình trạng lỗi lệch môi trường.¹

Demo Code - Dockerfile Đa lớp cho Product Service (Nền tảng Django):

```

FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

WORKDIR /app

COPY requirements.txt /app/requirements.txt
RUN pip install --no-cache-dir -r requirements.txt

COPY product_service/ /app/

EXPOSE 8000

```

Demo Code - Cấu trúc docker-compose.yml Hoàn chỉnh: Tập điều phối Compose này liên kết tất cả thành phần: Cấu trúc cơ sở dữ liệu Polyglot (Postgres, MySQL, Neo4j), Cổng điều hướng Nginx và các dịch vụ chức năng.

```
services:
  > Run Service
  mysql:
    image: mysql:8.4
    container_name: mysql
    environment:
      MYSQL_ROOT_PASSWORD: root
    volumes:
      - mysql_data:/var/lib/mysql
      - ./docker/mysql-init:/docker-entrypoint-initdb.d:ro
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "127.0.0.1", "-uroot", "-proot"]
      interval: 5s
      timeout: 5s
      retries: 20
      start_period: 20s
  > Run Service
  postgres:
    image: postgres:16
    container_name: postgres
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: admin
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./docker/postgres-init:/docker-entrypoint-initdb.d:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -h 127.0.0.1"]
      interval: 5s
      timeout: 5s
      retries: 20
      start_period: 20s
```

UI

